

Operational Governance & Versioning Document

Machine Learning Operations

Basit, Lepori, Pierro, Tomasi, Ziurkalov

February 12th 2026

Contents

1	Version Control Strategy	1
1.1	Branching Strategy	2
1.2	Code Versioning	2
1.3	Model Versioning	4
2	CI/CD and Automation Strategy	4
2.1	Continuous Integration	4
2.2	Continuous Deployment and Retraining	4
2.3	Promotion and Rollback Strategy	5
2.4	Future Automation Readiness	5
3	Model Lifecycle Governance	5
3.1	Model Registry	5
3.2	Reproducibility	6
3.3	Experiment Tracking	6
4	Monitoring and Maintenance	7
4.1	Monitoring Scope	7
4.2	Data Drift Detection	7
4.3	Incident Response and Escalation	7
4.4	Maintenance Plan	8

Version Control Strategy

A well-defined version control strategy is a cornerstone of operational governance in Machine Learning Operations. It enables structured collaboration among multidisciplinary teams, ensures traceability of changes, and guarantees reproducibility across code, data, and models.

In this project, version control is designed as an integrated governance mechanism covering source code, datasets, machine learning models, and documentation. The strategy explicitly separates concerns across these components while maintaining strong linkage between them to support auditing, debugging, and long-term system evolution.

Branching Strategy

The project adopts a controlled branching strategy designed to support iterative development while preserving a stable and auditable system state. Version control is managed through GitHub, with a single authoritative branch and a set of purpose-specific development branches.

The **main** branch represents the stable and validated state of the system. Only reviewed and consolidated changes are merged into **main**, ensuring that it always reflects a reproducible and integration-ready configuration.

Development activities are conducted on dedicated branches, each aligned with a specific functional area or experimental objective. Feature branches are used to isolate model development and experimentation, allowing parallel progress without compromising system stability.

Branch	Governance Role
main	Stable branch containing validated pipelines, models, and configurations approved for integration and documentation.
ML-pipeline	Branch dedicated to the development and refinement of the end-to-end machine learning pipeline, including training, evaluation, and orchestration logic.
feature/los-prediction-model	Feature branch focused on the implementation and tuning of the Length of Stay prediction model, including feature selection and model evaluation.
feature/model-retraining-with-feedback	Experimental branch exploring retraining strategies based on feedback loops and model performance degradation signals.
Data-Preprocessing	Branch dedicated to data ingestion, cleaning, pre-processing, and schema validation routines prior to integration into the ML pipeline.

Feature branches are merged into the **ML-pipeline** branch during development and subsequently promoted to **main** once validation criteria are satisfied. This staged promotion strategy enables controlled evolution of the system while maintaining traceability between development activities and the stable release state.

All branches remain accessible for audit and historical analysis, supporting debugging, experiment reproducibility, and governance review.

Code Versioning

Code versioning reflects the progressive evolution of the system and is tightly coupled with data availability and modeling maturity. Each code version corresponds to a clearly defined development stage and is tagged accordingly.

Code Version	Description
Version 1	Initial implementation of the data ingestion pipeline, process discovery, and overall system architecture.
Version 2	Introduction of data preprocessing logic, exploratory analysis, and baseline model implementation.
Version 3	Refinement of preprocessing steps and integration of bottleneck analysis and evaluation modules.
Version 4	Implementation of data versioning mechanisms and general codebase stabilization.

Code versions are immutable once released, enabling precise reconstruction of the system state associated with any experiment or model artifact.

Model Versioning

Model versioning is managed as a governed process ensuring traceability, reproducibility, and controlled evolution of predictive models. Each trained model is assigned a unique version identifier and explicitly linked to the corresponding code and data versions used during training.

Model development follows a progressive strategy, starting from simple baselines and advancing toward more robust models that balance predictive performance and clinical interpretability.

Each model version is immutable once validated and remains accessible for auditing, comparison, or rollback. By maintaining distinct identifiers for code, data, and models, the system can distinguish performance degradation caused by data drift from that caused by architectural or algorithmic limitations, supporting informed governance decisions.

CI/CD and Automation Strategy

Continuous Integration and Continuous Deployment (CI/CD) plays a critical role in the operational governance of machine learning systems, particularly in safety-critical domains such as Emergency Department operations, where data distributions and clinical workflows may evolve rapidly.

In this project, CI/CD is conceived as a governance-aware automation framework rather than a fully autonomous production deployment system. While end-to-end automated deployment is not enabled in the current implementation, the system architecture is explicitly designed to be CI/CD-ready and compatible with future automation extensions.

Continuous Integration

Continuous Integration focuses on ensuring that incremental changes to code, data processing logic, and modeling components do not compromise system integrity. Integration is governed through controlled merge operations into the `main` branch, following validation and review.

CI practices include:

- automated execution of pipeline scripts in a containerized environment to verify reproducibility,
- validation of data preprocessing and feature engineering steps,
- consistency checks ensuring alignment between code versions, dataset snapshots, and model configurations.

These integration checks reduce the risk of silent failures and enforce a reproducible baseline at every stable system state.

Continuous Deployment and Retraining

Continuous Deployment is treated as a staged and controlled process. Rather than automatically deploying new models to a live clinical environment, model promotion follows an explicit validation and governance workflow.

The system is designed to support automated retraining triggers, including:

- detection of sustained performance degradation on recent data,
- accumulation of prediction errors beyond predefined thresholds,
- availability of new validated data snapshots.

When such conditions are met, a retraining process can be initiated in batch mode. Candidate models produced during retraining are evaluated against established acceptance criteria before being promoted.

Promotion and Rollback Strategy

Model promotion is conditional upon successful validation and compliance with predefined performance and stability requirements. Newly trained models are registered as candidate versions and compared against the currently active model.

Only models that demonstrate measurable improvement or equivalent performance with improved robustness are eligible for promotion. Previous validated models remain accessible and can be reinstated if performance regressions or unexpected behaviors are observed, enabling controlled rollback.

Future Automation Readiness

Although full CI/CD automation is not activated, the system architecture—including containerized execution, versioned artifacts, and modular training scripts—ensures compatibility with future CI/CD pipelines.

This design enables seamless integration with automated orchestration frameworks without requiring architectural changes, supporting long-term scalability and governance.

Model Lifecycle Governance

Model Lifecycle Governance defines the policies, processes, and controls required to manage machine learning models from initial development to validation, deployment, monitoring, and eventual retirement. In safety-critical and data-intensive domains such as healthcare operations, governance is essential to ensure reproducibility, traceability, and controlled model evolution.

In this project, model lifecycle governance is implemented as an integrated framework that explicitly links trained models to the data, code, and configuration used to produce them. This approach ensures that every model artifact is auditable, reproducible, and comparable over time.

Model Registry

The Model Registry serves as the authoritative repository for all trained and validated models. Each model version is stored as an immutable artifact and is uniquely identified by its model version, training configuration, and associated data snapshot.

Once a model is registered, it cannot be modified. This immutability guarantees that historical models remain available for audit, comparison, rollback, or re-evaluation under new operational conditions.

Registered models are associated with explicit lifecycle states, including:

- **Development:** models generated during experimentation and preliminary evaluation.
- **Validated:** models that satisfy predefined performance and stability criteria.
- **Archived:** historical models retained for traceability and governance purposes.
- **Deprecated:** models that are no longer recommended for use due to inferior performance or changed operational requirements.

This state-based model management enables controlled promotion and rollback while preserving a complete history of model evolution.

Reproducibility

Reproducibility is enforced as a fundamental governance requirement. Every registered model can be exactly reproduced by re-executing the training process under the same conditions.

Reproducibility is ensured through the explicit linkage of:

- **Versioned datasets:** immutable data snapshots used as training input.
- **Versioned code:** tagged code versions defining preprocessing, feature engineering, and model training logic.
- **Training configurations:** serialized hyperparameters, random seeds, and execution settings.

This triad guarantees that model behavior can be traced back to its precise generation context, supporting debugging, validation, and regulatory accountability.

Experiment Tracking

Experiment tracking provides visibility into model behavior across multiple training runs and supports informed governance decisions during model selection and promotion.

Each experiment records:

- training and validation metrics,
- execution metadata (e.g., runtime, resource usage),
- references to the corresponding code version and data snapshot.

Tracked experiments enable systematic comparison between candidate models and facilitate the identification of performance trends, regressions, and improvements over time. This structured experimentation process ensures that model promotion decisions are based on empirical evidence rather than ad-hoc evaluation.

Monitoring and Maintenance

Long-term reliability, safety, and accuracy are essential requirements for machine learning systems supporting Emergency Department operations. To meet these requirements, models must be continuously supervised and maintained throughout their operational lifecycle rather than treated as static artifacts.

In this project, monitoring and maintenance are designed as complementary governance processes addressing both system-level performance and model-level stability. This approach ensures early detection of anomalous behavior and enables timely corrective actions.

Monitoring Scope

Monitoring activities cover two primary dimensions:

- **System performance:** execution time, pipeline stability, and data ingestion reliability.
- **Model behavior:** prediction accuracy, error distributions, and temporal consistency of outputs.

These dimensions jointly support the detection of operational risks and performance degradation.

Data Drift Detection

Data drift represents a significant risk in healthcare-related machine learning systems, as changes in patient populations, clinical procedures, or external events (e.g., pandemics or large-scale emergencies) can substantially alter data distributions.

Drift detection is implemented by continuously comparing model predictions against observed outcomes recorded during routine operations. The monitoring framework focuses on identifying systematic deviations rather than isolated prediction errors.

Key drift indicators include:

- **Monitored variables:** sustained changes in predicted and observed waiting times across patient encounters.
- **Warning signals:** deviations from expected error distributions or statistical thresholds trigger monitoring alerts.
- **Trigger conditions:** retraining is considered when prediction errors exceed predefined thresholds over a sustained observation window.

This strategy enables early detection of concept or data drift while minimizing false alarms caused by short-term fluctuations.

Incident Response and Escalation

When monitoring signals indicate abnormal behavior, a structured response process is initiated. Alerts generated by drift detection or system monitoring are reviewed to assess severity and potential impact.

Depending on the nature of the issue, corrective actions may include:

- initiating conditional model retraining,
- reverting to a previously validated model version,
- investigating data quality or pipeline execution failures.

This escalation framework ensures that corrective actions are traceable, controlled, and aligned with governance policies.

Maintenance Plan

The maintenance plan defines a set of automated and conditional tasks designed to minimize manual intervention while preserving system robustness and performance over time. Maintenance activities are executed according to predefined schedules or triggered by monitoring signals.

Maintenance Task	Execution Type and Frequency
Data Drift Detection	Automated, continuous monitoring
System Performance Monitoring	Automated, continuous
Data Quality Validation	Automated, upon data ingestion
Conditional Model Retraining	Triggered by sustained performance degradation
Scheduled Model Retraining	Automated, periodic (e.g., bi-monthly)

All maintenance actions are logged and traceable, ensuring transparency and supporting auditability. Together, monitoring and maintenance activities establish a governance framework that supports safe, reliable, and adaptive model operation over time.